

Programs:

P#1 Write a program to add 2 number using indirect addressing

DATA SEGMENT

NUM1 DW 5 ; first number

NUM2 DW 3 ; second number

RESULT DW ? ; to store result

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START:

MOV AX, DATA ; initialize data segment

MOV DS, AX

LEA SI, NUM1 ; load address of NUM1 into SI

MOV AX, [SI] ; move value at address SI (NUM1) into AX

LEA DI, NUM2 ; load address of NUM2 into DI

ADD AX, [DI] ; add value at address DI (NUM2) to AX

LEA BX, RESULT ; load address of RESULT into BX

MOV [BX], AX ; store the sum in RESULT

MOV AH, 4CH ; terminate program

INT 21H

CODE ENDS

END START

P#2:Write an assembly language program to add four number using byte variable. The program should store four numbers in memory, load them into registers one by one, perform addition, and finally store the result (sum) back into memory.

DATA SEGMENT

NUM1 DB 10H ; first number (16 in decimal)
NUM2 DB 20H ; second number (32 in decimal)
NUM3 DB 15H ; third number (21 in decimal)
NUM4 DB 05H ; fourth number (5 in decimal)
SUM DB ? ; to store the result

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START:

MOV AX, DATA ; initialize data segment
MOV DS, AX

MOV AL, NUM1 ; load first number into AL
ADD AL, NUM2 ; add second number
ADD AL, NUM3 ; add third number
ADD AL, NUM4 ; add fourth number

MOV SUM, AL ; store the result in memory (SUM)

```

MOV AH, 4CH    ; terminate program
INT 21H

CODE ENDS

END START

```

P#3:Write a program to search a particular element from an array using binary search. if the element is found set AX to one and otherwise to zero.

```

DATA SEGMENT

ARR DB 2, 5, 8, 12, 16, 23, 38, 56, 72, 91 ; sorted array (10 elements)

N   DB 10                      ; number of elements

KEY DB 23                      ; element to search

FOUND DB ?                     ; to store result (1 or 0)

DATA ENDS

```

```

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

```

```

START:

MOV AX, DATA    ; initialize data segment
MOV DS, AX

; Initialize pointers and registers

XOR AX, AX

MOV BL, 0        ; low = 0

MOV BH, N        ; high = N

DEC BH           ; high = N - 1

```

SEARCH_LOOP:

CMP BL, BH

JG NOT_FOUND ; if low > high, element not found

MOV AL, BL

ADD AL, BH

SHR AL, 1 ; mid = (low + high) / 2

MOV SI, OFFSET ARR

ADD SI, AX ; address of ARR[mid]

MOV DL, [SI] ; DL = ARR[mid]

CMP DL, KEY

JE FOUND_LABEL ; if equal → found

JB LESS_THAN_KEY ; if ARR[mid] < KEY

JA GREATER_THAN_KEY

LESS_THAN_KEY:

MOV BL, AL

INC BL

JMP SEARCH_LOOP

GREATER_THAN_KEY:

MOV BH, AL

DEC BH

JMP SEARCH_LOOP

FOUND_LABEL:

MOV AX, 1 ; AX = 1 means found

MOV FOUND, 1

JMP END_PROGRAM

NOT_FOUND:

MOV AX, 0 ; AX = 0 means not found

MOV FOUND, 0

END_PROGRAM:

MOV AH, 4CH

INT 21H

CODE ENDS

END START